



ORBIS: AN INTELLIGENT ACADEMIC SUPPORT SYSTEM WITH AI AND RAG

by

ATAKAN GÜL, 121200152

ARDA KAAAN Y2LD2Z, 122200045

Supervised by

ÖZGÜR ÖZDEMİR

Submitted to the

Faculty of Engineering and Natural Sciences
in partial fulfillment of the requirements for the

Bachelor of Science

in the

Department of Computer Engineering

June, 2026

Abstract

University academic systems often function merely as static record-keeping databases, lacking proactive support for students navigating complex degree requirements. This project, 'Orbis', introduces an intelligent academic support system designed to bridge this gap using Artificial Intelligence (AI) and Retrieval-Augmented Generation (RAG). The core of Orbis is an Orchestrator-led event management pipeline that autonomously monitors a student's dynamic academic state against a complex vector database of university regulations, triggering timely notifications for critical milestones. To achieve high fidelity, we engineered a taxonomy generation pipeline utilizing Shannon entropy tail-signal classification and MiniBatchKMeans clustering, organizing 60,649 document chunks into 125 high-quality leaf nodes. Our pipeline features a two-pass adversarial LLM review, evaluating 323 candidate rules and extracting 178 highly actionable obligations with a 55% acceptance rate, and matches them against 4 distinct student profiles to generate 176 personalized assignments. We evaluate each subsystem against a gold ground-truth set manually validated by the authors. The event-extraction stage achieves precision 0.978 / recall 0.574 / F_1 0.723 over 323 candidates; the assignment-matching stage achieves F_1 0.51 over 712 (profile, obligation) pairs. An agentic submission-review subsystem evaluated on 32 deterministically generated cases attains 93.8% accuracy with perfect recall and full (4/4) resistance to in-document prompt injection. Complementing the event and submission systems, a production-grade RAG chatbot enables natural-language queries over the full institutional knowledge base; evaluated against a 100-query bilingual benchmark across seven routing categories, it achieves 92.3% routing accuracy, 94.7% context recall, a Mean Reciprocal Rank of 0.94, and 97.6% semantic accuracy. The remaining roadmap includes direct integration with the university's Student Information System (SIS) and injection-hardened submission review.

TABLE OF CONTENTS

Abstract	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
1 Introduction	1
2 Related Works	2
3 Design	5
3.1 Dataset	5
3.1.1 RAG Knowledge Base Preparation	6
3.2 Theoretical Background	8
3.2.1 Shannon Entropy for URL Classification	8
3.2.2 Vector Embeddings and Cosine Similarity	8
3.2.3 MiniBatchKMeans and Ward HAC	9
3.2.4 Hybrid Retrieval and Neural Re-Ranking	9
4 Methodology	10
4.1 Data Categorization & Taxonomy Strategy	10
4.2 The Orchestrator-Led Event Pipeline	11
4.3 Contextual vs. SQL Assignment Matching	12
4.4 Agentic Assignment Submission Evaluation	13
4.5 RAG Chatbot Architecture	15
4.5.1 End-to-End Query Walkthrough	15
4.5.2 Knowledge Base Construction and Ingestion Pipeline .	19

4.5.3	LLM-Based Query Routing	19
4.5.4	Hybrid Search with Two-Stage Neural Re-Ranking . .	20
4.5.5	Context Building and Streaming Response Generation	21
5	Experimental Setup	22
5.1	Framework and Deployment	22
5.2	Performance Metrics	23
5.3	RAG Chatbot System Configuration	23
5.4	RAG Evaluation Dataset	24
5.5	RAG Evaluation Metrics	24
6	Experiments & Discussion	25
6.1	Extraction Throughput & Quality Funnel	25
6.2	Assignment Generation Coverage	25
6.3	Gold Ground-Truth Construction	27
6.4	Event Extraction Results	28
6.5	Assignment Matching Results	29
6.6	Submission Agent Results	29
6.7	RAG Chatbot Evaluation	31
6.8	Limitations	33
7	Conclusion and Future Work	33
	References	35
	Appendix	37
A.1	RAG Router Prompt	37
A.2	RAG Generative Agent Prompt	38

LIST OF FIGURES

1	Taxonomy Pipeline Architecture	11
2	Proactive Event Pipeline Architecture	12
3	Agentic Submission-Evaluation Flow	14
4	Chatbot Query Input	15
5	RAG Chatbot Pipeline	18
6	Chatbot Response Output	22

LIST OF TABLES

1	Positioning of Orbis Against Related Systems	4
2	Baseline and Comparator Validity	5
3	Dataset and Post-Processing Statistics	6
4	Web Page Corpus Distribution	7
5	RAG Evaluation Benchmark Categories	24
6	Event Extraction Pipeline Funnel	25
7	Assignments per Student Profile	26
8	Metric Provenance and Limitations	28
9	Event Extraction Confusion Matrix	28
10	Event Extraction Metrics	28
11	Per-Profile Assignment Matching Scores	29
12	Submission Agent Results by Stratum	30
13	Submission Agent Confusion Matrix	30
14	Submission Agent Metrics	30
15	Submission Agent Ablation Study	31
16	RAG Chatbot Evaluation Results	31

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration/Continuous Deployment
DB	Database
GIN	Generalized Inverted Index
HAC	Hierarchical Agglomerative Clustering
HNSW	Hierarchical Navigable Small World
LLM	Large Language Model
MRR	Mean Reciprocal Rank
RAG	Retrieval-Augmented Generation
SIS	Student Information System
SSE	Server-Sent Events
TF-IDF	Term Frequency-Inverse Document Frequency

1 Introduction

In universities, students face a significant cognitive load managing complex degree rules, course selections, and deadlines. Traditional academic platforms are often designed as static databases. They store information efficiently but fail to interpret it for the student’s benefit. Students generally do not miss deadlines because they cannot search a system; they miss them because they do not know what they are supposed to be looking for. This creates a “synchronization gap” between student needs and institutional data.

Existing AI-mediated support tools only partially close this gap. Retrieval-Augmented Generation grounds a generative model’s answers in source documents and substantially reduces hallucination [1], and a growing body of advising-chatbot research applies it to student questions. However, a recent systematic review of advising chatbots found that most deployed systems remain fundamentally reactive: they answer a question once a student thinks to ask it, rather than enforcing obligations against the student’s state [2]. In parallel, regulatory information-extraction research has shown that obligations and requirements can be parsed out of dense legal and policy text with high precision [3, 4], but this capability has rarely been connected to a student-facing advising system.

Our project, ‘Orbis’, offers a novel solution by shifting the academic paradigm from static storage to autonomous orchestration. Orbis is built to act as a proactive academic agent operating through two tightly integrated subsystems. The **Proactive Event System** autonomously evaluates a student’s specific academic state against unstructured university regulations to trigger unprompted, obligation-based notifications. The **RAG Chatbot** complements this by enabling students and faculty to interactively query the same institutional knowledge base in natural language; answering questions about regulations, course catalogs, academic calendars, and personal timetables in real time. While standard AI integrations in education wait for user prompts, the core innovation of Orbis is its autonomous evaluation of a student’s specific state against unstructured, complex university regulations.

Crucially, the architecture of Orbis is designed for scalability. The system relies on a strictly orchestrated workflow to execute its primary use case: the Proactive Event System. This autonomous pipeline extracts strict, assignable obligations from regulatory documents and creates normalized events, allowing the system to trigger timely, unprompted notifications for students regarding their specific academic requirements.

The rest of this report is organized as follows. Section 2 reviews related works. Section 3 presents the system design, covering the institutional dataset and the theoretical foundations. Section 4 details the methodology of each subsystem, including the proactive event pipeline, the submission agent, and the RAG chatbot. Section 5 describes the experimental setup. Section 6 reports the experiments, results, and discussion across all subsystems. Section 7 concludes the report and outlines future work. The router and generation prompts that drive the RAG chatbot are reproduced in the Appendix.

2 Related Works

Academic support systems have evolved from static record-keeping into AI-mediated tools, but most existing systems remain reactive: they answer student questions but do not autonomously enforce regulations against a student’s state. We position Orbis against six lines of work.

RAG-based academic chatbots. Retrieval-Augmented Generation combines a generative LLM with a non-parametric retrieval store, grounding answers in source documents and reducing hallucination [1]. A systematic review of advising chatbots screened 128 studies and retained 11 advising-chatbot papers, finding that most systems target admission or academic-advising dialogue rather than autonomous obligation enforcement [2]. More recent advising-LLM work reports QA-style metrics such as BLEU, ROUGE-L, and student satisfaction ratings [5]. These systems serve well as on-demand search interfaces, but operate strictly in a pull mode: a student must already know that a question is worth asking. Orbis instead operates in a push mode: it parses regulations into structured obligations and matches them against profiles before the student asks.

Degree-audit systems. Tools such as DegreeWorks and Stellic encode degree requirements as deterministic rules and audit a transcript against them. The Degree Audit Reporting System (DARS), for example, has been studied as an advising and registration-support system rather than as an information-extraction benchmark [6]. Such systems are excellent for objective conditions (credit counts, GPA thresholds) but cannot reason about unstructured policy text (procedural requirements, deadline-bearing obligations, exceptions written in prose). Orbis complements deterministic audit with a contextual LLM path that operates over free-text regulation.

Regulatory information extraction. Prior regulatory IE work is closer to Orbis’s event-extraction subsystem, but not to the full Orbis pipeline.

Zhang and El-Gohary report 0.969 precision and 0.944 recall when extracting quantitative requirements from the 2009 International Building Code using semantic NLP rules [3]. Recent legal-obligation extraction work on the EU AI Act reports 93% precision for obligation filtering and over 99% accuracy for classifying obligation type, addressee, and predicate [4]. These results are useful precision/recall comparators for the extraction stage, but their domains, source documents, and output schemas differ from university-student obligation assignment.

Local context: HermiOne. Istanbul Bilgi University has deployed “HermiOne,” a support chatbot built by the university’s IT department. Based on publicly observable behavior at the time of writing, HermiOne appears to operate primarily as a reactive question-answering chatbot rather than as a proactive regulation-enforcement agent; a direct comparison would require access to its design documentation, which we did not have. Orbis is intentionally complementary: HermiOne handles the conversational interface, while Orbis produces the underlying structured event stream that a future iteration of HermiOne could subscribe to.

Hybrid retrieval and neural re-ranking. A consistent finding in the RAG literature is that dense vector search alone is insufficient when the target corpus is large and heterogeneous: exact-term queries benefit disproportionately from sparse keyword retrieval. Hybrid approaches that interleave keyword (inverted-index) results with dense vector results before a final re-ranking step have been shown to outperform either method in isolation across domain-specific corpora [7]. Beyond retrieval, cross-encoder re-rankers — which score (query, document) pairs jointly rather than encoding them independently — substantially improve precision when applied to the candidate pool before it reaches the generative model [8]. The Orbis RAG chatbot implements both of these findings as load-bearing components rather than optional optimizations.

Automated submission assessment. LLM-assisted grading systems provide a useful comparison point for the submission-review subsystem, but their metrics usually measure grade prediction rather than safety-oriented accept/reject screening. JorGPT, for instance, evaluates programming-assignment grading over 672 submissions and reports adjusted $R^2=0.9156$, mean absolute error 0.4579, and a reduction from over 300 hours of manual work to under 15 minutes of automated processing [9]. Orbis’s submission agent is narrower: it does not assign grades, but checks whether a submitted artifact satisfies assignment requirements and preserves an appeal path.

LLM-as-a-judge for evaluation. Because labeled gold sets for university-specific regulation extraction do not exist publicly, we adopt the LLM-as-a-judge methodology [10], in which an independent LLM is used as a scalable proxy for human labeling and validated against a second model from a different family. This methodology underlies our evaluation in Section 6.

Table 1 summarizes the comparison.

System type	Pull/push	Unstructured rules	Student-specific	Auto-notify
University FAQ chatbot	Pull	No	No	No
RAG academic chatbot	Pull	Yes	No	No
Deterministic degree audit	Push	No	Yes	Partial
HermiOne (reactive)	Pull	Yes	Partial	No
Orbis (this work)	Push	Yes	Yes	Yes

Table 1: Positioning of Orbis against related categories of academic-support systems. *Push* means the system surfaces an obligation before a student asks. *Unstructured rules* means handling free-text regulation, not only schema-based requirements.

No exact end-to-end baseline was found for the combined task Orbis addresses: proactive extraction of university regulations, profile-specific obligation assignment, and submission review with appeal. Therefore, Table 2 compares Orbis against official prior work at the subsystem level and explicitly marks whether each comparison is direct, partial, or contextual. This distinction is important: we do not claim that Orbis “beats” systems evaluated on different tasks; we use them to position the contribution and to choose defensible metrics.

Official prior work	Related Orbis subsystem	Reported metric or scope	Comparison type	How it is used in this report
Assayed et al. systematic review [2]	Advising-chatbot landscape	128 articles screened; 11 included advising-chatbot studies	Contextual	Shows that advising chatbots are active research systems, but mostly dialogue/admission/advising tools rather than proactive rule-assignment pipelines.
Ismail academic-advising LLM [5]	Advising chatbot	BLEU 0.45→0.78; ROUGE-L 0.85; 50-student ratings 4.6–4.8/5	Contextual	Provides QA-chatbot metrics; not directly compared because Orbis is evaluated on extraction, matching, and submission screening rather than answer generation.
Wick and Wallingford DARS study [6]	Degree audit	Qualitative analysis of DARS for advising and registration support	Contextual	Positions deterministic degree-audit systems as complements, not direct NLP baselines.
Zhang and El-Gohary regulatory IE [3]	Event extraction	Precision 0.969; recall 0.944 on building-code quantitative requirements	Partial	Provides a precision/recall comparator for regulatory extraction, with different domain and rule schema.
Galli et al. AI Act obligation extraction [4]	Event extraction	93% obligation-filtering precision; >99% classification accuracy for extracted obligation fields	Partial	Provides a modern legal-obligation extraction comparator; not compared to assignment matching or submission review.
JorGPT LLM grading [9]	Submission assessment	Adjusted R^2 0.9156; MAE 0.4579; 672 submissions; 300+ hours → under 15 minutes	Partial	Provides grading/workload context; Orbis reports binary accept/reject safety metrics instead of grade-regression metrics.
Prior Orbis single-call reviewer	Submission review	Accuracy 0.781; injection resistance 1/4	Direct	Internal ablation baseline for the current two-call reviewer reported in Section 6.6.

Table 2: Official baseline and comparator validity. *Direct* means the same task and data family; *partial* means a related subsystem metric but different domain or output schema; *contextual* means useful for positioning only.

3 Design

This section establishes the foundations on which the Orbis system is designed: the institutional dataset that every subsystem operates on, and the theoretical tools that underpin the data-categorization, retrieval, and clustering machinery described later in Section 4.

3.1 Dataset

The foundation of Orbis is its institutional `knowledge_base`, constructed from 80 official source URLs under the `bilgi.edu.tr` domain. The dataset and its post-processing artifacts are summarized in Table 3.

Quantity	Value
Source URLs crawled	80
Raw document chunks	60,649
Languages observed	English, Turkish
Top-level taxonomy categories (LLM-mapped)	11
Leaf-level taxonomy nodes	125
Regulatory leaves (post-overlay)	29
Regulatory chunks (event-pipeline input)	935
Accepted obligations (<code>regulation_rules</code>)	178
Student profiles (<code>user_profiles</code>)	4

Table 3: Dataset and post-processing statistics. The Proactive Event System operates only on the 935 regulatory chunks; all other categories are routed to the RAG search index.

The crawl is intentionally heterogeneous: it contains formal regulations (student handbooks, dual-major directives, internship procedures), administrative announcements, event pages, and upload directories. The corpus is bilingual, which complicates deduplication — the same regulation often exists under both `/en/academics/...` and `/tr/akademik/...` paths. Chunking is performed at ingest time using a semantic splitter with a soft 800-token target and a 100-token overlap; each chunk is embedded once with a local MiniLM model and stored in `knowledge_base_embeddings` alongside the source URL, content hash, and categorical metadata. The regulation subset is isolated via the four-step taxonomy pipeline of Section 4.1, with a final regex overlay that promotes any chunk whose source URL matches a known regulation path into the `regulation` or `regulation_document` category, preventing event-pipeline cross-contamination from non-binding content. Instead, evaluation is performed against a constructed Gold Ground-Truth testing set, detailed in Section 6.3.

3.1.1 RAG Knowledge Base Preparation

The same `knowledge_base` table serves as the retrieval index for the RAG chatbot, but the full chatbot-facing corpus has a substantially broader scope than the 935-chunk regulatory subset consumed by the event pipeline. The chatbot index is built from web pages and PDFs split into **150-word chunks with a 30-word overlap**, hard-capped to remain within the embedding

model’s token limit. Understanding the corpus structure is necessary to explain several design decisions in the retrieval architecture.

Web page distribution. Approximately 12,600 web pages were scraped from the university domain. Table 4 shows the category breakdown.

Category	Count	Share	Description
News & Announcements	≈4,160	33%	Press releases, news items (/haber/)
Events	≈4,035	32%	Past and future seminars (/etkinlik/)
Academic (Faculties)	≈2,268	18%	Department pages, faculty bios, course lists
University Info	≈1,008	8%	Rules, administration, “About Us” pages
Student Life	≈630	5%	Transport, clubs, support services
Other	≈507	4%	International office, research centers

Table 4: Web page corpus distribution by category.

65% of the corpus is *temporal* content (news and event pages), while only ≈35% is *core knowledge*. This imbalance is the primary retrieval challenge: a naïve vector search for “scholarship policy” risks surfacing a 2018 news article announcing a policy update rather than the current policy page. The router architecture (Section 4.5) addresses this by steering factual queries away from high-temporal URL prefixes.

PDF filtering. Approximately 2,245 scraped PDFs were analyzed. Construction tenders (/ihaleler/) use legalistic language (“Article 5”, “Obligations”) that semantically overlaps with academic regulations; recruitment result lists contain thousands of proper nouns and scores that dilute the vector space. A `filter_pdfs.py` script applied hard-exclusion rules by URL pattern and Turkish keyword (e.g. “teklif mektubu”—Proposal Letter; “sonuç listesi”—Result List), with hard-inclusion overrides protecting known high-value documents (any file containing “erasmus”, “staj”, “handbook”, or “yönerge” is always retained). The result: **1,595 files ingested** (regulations, syllabuses, Erasmus guides, brochures), **315 discarded** (tenders, outdated recruitment lists), and **335 archived** (faculty CVs, reserved for a future Faculty Search feature).

Language detection. Accurate language tagging drives correct PostgreSQL stemming in the keyword search index. A simple character-frequency heuristic (counting Turkish-specific characters such as `ı`, `ğ`, `ş`) failed on English documents containing local addresses (e.g., “Santral Campus, Eyüpsultan”). The adopted approach runs a *Stopword Race*: it counts occurrences of language-specific stopwords (*the*, *and*, *is* vs. *ve*, *bir*, *için*) over the document body, resolving language from content rather than character inventory. A `fix_pdf_language_and_titles` preprocessing step repaired **361 PDF titles** (22.6% of the corpus) that carried generic software-generated names (e.g., “PowerPoint Presentation”), falling back to a clean title derived from the filename. Final language split: $\approx 65\%$ Turkish, $\approx 35\%$ English.

3.2 Theoretical Background

To successfully isolate and evaluate this dataset, Orbis relies on several foundational machine learning and information theory algorithms.

3.2.1 Shannon Entropy for URL Classification

To classify opaque URL structures, we measure the diversity of URL “tails” using Shannon Entropy [11]. A high entropy value indicates that a URL directory is semantically informative, whereas low entropy implies repetitive or dynamic templates.

$$H = - \sum_{i=1}^n p_i \cdot \log_2(p_i)$$

Where p_i is the probability of a specific URL pattern occurring within a given parent group.

3.2.2 Vector Embeddings and Cosine Similarity

Text chunks are transformed into high-dimensional numerical vectors. To measure the semantic similarity between two chunks (or a user query and a document), we calculate the Cosine Similarity.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

Prior to clustering, we perform L2-Normalization on all vectors ($\hat{v} = \frac{v}{\|v\|}$), converting cosine distance into Euclidean distance on a unit sphere, optimizing it for K-Means processing.

3.2.3 MiniBatchKMeans and Ward HAC

For semantic discovery of opaque URLs, we deploy MiniBatchKMeans, which utilizes random mini-batches to achieve linear time complexity $\mathcal{O}(n)$ suitable for large datasets [12]. This is followed by Ward Hierarchical Agglomerative Clustering (HAC) on the centroids. Ward’s method merges clusters to minimize the total within-cluster variance [13].

3.2.4 Hybrid Retrieval and Neural Re-Ranking

The RAG chatbot relies on two complementary retrieval mechanisms executed in parallel before a cross-encoder re-ranking stage consolidates their outputs.

Full-text keyword search operates on a TSVECTOR column maintained by PostgreSQL as a `GENERATED ALWAYS AS ... STORED` computed expression. The column applies language-aware stemming — automatically selecting the Turkish or English lexical parser based on each row’s `language` field — to both the document title (indexed at weight ‘A’, higher priority) and the content body (weight ‘B’). Queries are converted to `tsquery` expressions using OR logic, ensuring that any single key-term match surfaces the document. A GIN (Generalized Inverted Index) is maintained on this column to enable sub-millisecond full-text lookups at 60,000+ rows.

Semantic vector search queries a 384-dimensional `pgvector` embedding column using L2/Euclidean distance against an HNSW (Hierarchical Navigable Small World) graph index [14]. HNSW enables approximate nearest-neighbor search with a configurable recall/speed trade-off; the L2 metric is an intentional implementation choice that performs well empirically with the selected MiniLM embedding model.

Cross-encoder re-ranking applies a neural model (`jina-reranker-v2-base-multilingual`) that scores each (query, document) pair *jointly*, capturing relevance nuances that a bi-encoder — which encodes query and document independently into fixed vectors — cannot. Cross-encoders are computationally expensive and therefore impractical for initial retrieval over large corpora, but highly effective when applied to the smaller candidate pool produced by hybrid search. The Orbis chatbot applies this step *twice* in a two-stage architecture described in Section 4.5.

4 Methodology

The Orbis methodology spans five components: the data-categorization (taxonomy) pipeline, the Orchestrator-led event pipeline, the contextual/SQL assignment matcher, the agentic submission reviewer, and the RAG chatbot.

4.1 Data Categorization & Taxonomy Strategy

Standard RAG systems suffer degraded accuracy when mixing formal rule-books with informal forum posts. Orbis mitigates this via a four-step semantic discovery pipeline.

1. **Step 1: Rule Engine:** Evaluates URL prefixes calculating semantic and dynamic ratios. Groups with high dynamic ratios (e.g., UUIDs) are rejected.
2. **Step 2: Tail Signal Classification:** Computes the Shannon entropy of URL tails. High entropy groups are classified as KNOWN.
3. **Step 3: Semantic Discovery:** The 21,324 UNKNOWN documents undergo L2-normalization and MiniBatchKMeans clustering. Ward HAC generates 52 fine-level labeled clusters based on TF-IDF scoring.
4. **Step 4: AI Taxonomy Mapping:** An LLM maps the deduplicated groups into a closed vocabulary of 11 top-level categories and 96 specific leaf nodes. A targeted regex overlay isolates the 935 regulation chunks into 29 distinct regulatory leaves.

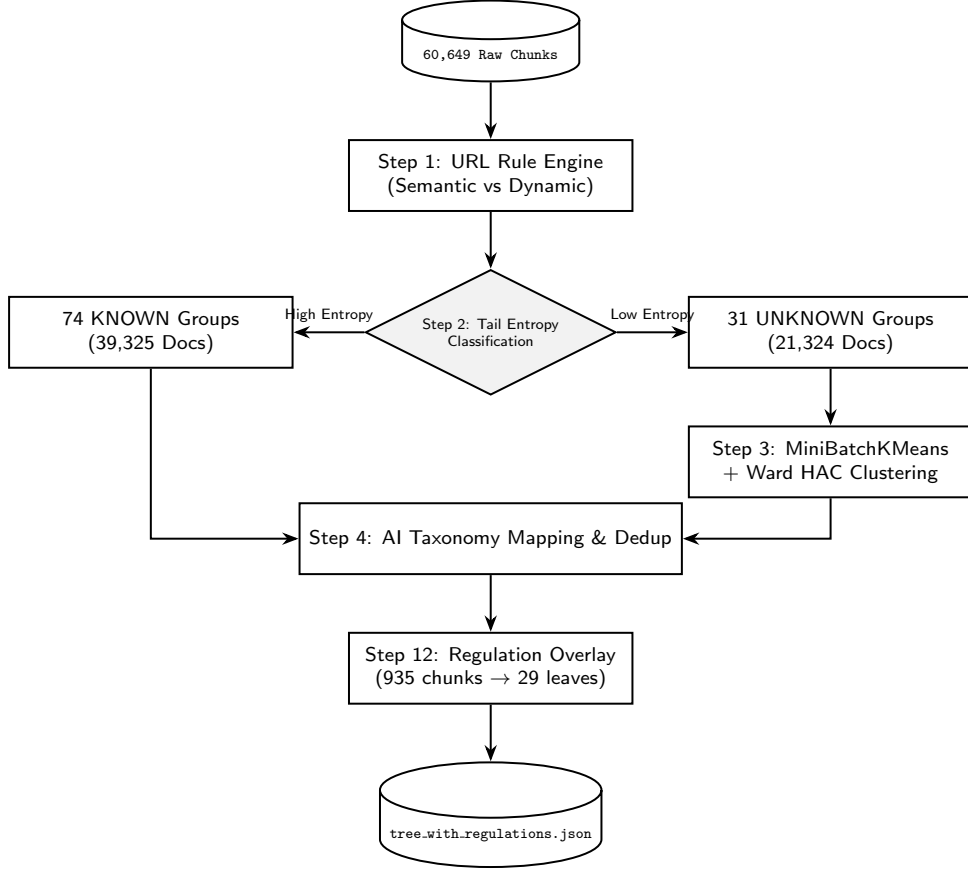


Figure 1: Data categorization and taxonomy pipeline architecture.

4.2 The Orchestrator-Led Event Pipeline

Unlike decentralized multi-agent systems, Orbis utilizes a centralized `EventPipelineOrchestrator`. This stateful hub controls stateless worker agents (`SearchAgent`, `ReasoningAgent`, `EventCreator`) to ensure database consistency and pipeline idempotency.

Figure 2 summarizes the proactive event pipeline. The system, led by a central orchestrator, leverages specialized agents to locate sources, extract candidates, validate them, and finally store the parsed, unambiguous rules into the database while maintaining rigorous execution states.

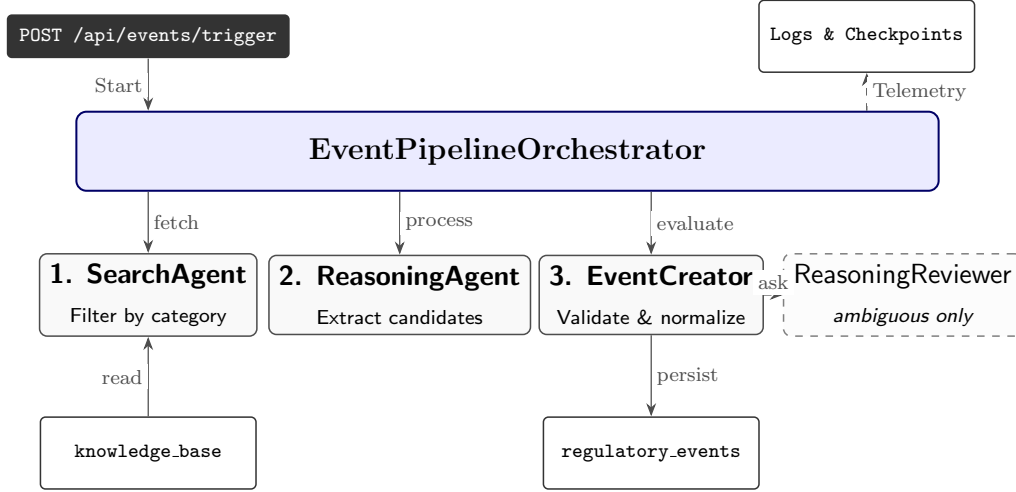


Figure 2: Sequential flow and architecture of the Orbis Proactive Event Pipeline. The orchestrator owns all run state; workers are stateless and communicate only via the orchestrator. The reviewer is invoked only for ambiguous candidates.

To prevent non-actionable events from reaching students, extraction follows a **Two-Pass LLM Adversarial Review**:

1. **Pass 1 (Extraction)**: The model evaluates full documents to generate candidate rules, defining triggers, deadlines, and target roles.
2. **Pass 2 (Adversarial Fix)**: The model reviews the original document alongside Pass 1 output to actively remove stale items, fix weak wording, and add missed rules.

4.3 Contextual vs. SQL Assignment Matching

User assignments are routed through two parallel channels based on the user’s `extra_context` JSON profile:

- **SQL Path (Deterministic)**: Objective thresholds (e.g., GPA constraints) are evaluated instantly via Python logic against database variables.
- **Contextual Path (LLM Reasoned)**: Complex rules are evaluated against the student’s text profile in batches. The LLM determines applicability and generates a human-readable justification.

4.4 Agentic Assignment Submission Evaluation

Beyond proactively assigning obligations, Orbis also evaluates work students submit against those assignments. Each upload is processed by a stateless `submission_agent` that, rather than issuing a single opaque verdict, reasons in causal steps that mirror how a human teaching assistant screens a submission: first understand *what is being asked*, then check the document *against each individual requirement* with cited evidence, and only then decide. Concretely the agent runs a deterministic file gate followed by a two-call LLM reasoning chain.

1. **File inspection (no LLM).** The agent records the file’s extension, size, MIME type, and — for archives — the member list. Empty files, oversized files, archives over a configured member limit, and unreadable or unsupported payloads are rejected here without ever invoking the LLM.
2. **Text extraction.** Supported formats (`.pdf`, `.docx`, `.zip`, and ~ 70 text-like extensions) are decoded to plain text. Inside archives the agent no longer trusts the extension as a gate: it skips only known-binary types and otherwise attempts to decode each member, keeping it if a printable-character heuristic confirms it is text (so a `.tex` project or an extensionless `Makefile` is still read).
3. **Requirement decomposition (LLM call 1).** The agent prompts the LLM to break the assignment title and description into a checklist of discrete, independently checkable requirements. If this call yields nothing usable, the agent falls back to a deterministic keyword `line_review` derived from the assignment text so the next step always has structure to anchor on.
4. **Per-requirement judgment and decision (LLM call 2).** The agent prompts the LLM with the requirement checklist, the file profile, and the extracted text (clipped to a configured budget), and asks it to walk *each* requirement: does the document satisfy it, with a quoted evidence span and a causal justification of *why*. Only after every requirement is judged does the model synthesize the overall `approved/rejected` decision. The output is a structured JSON report containing a `requirement_findings` array (one `{satisfied, evidence, evidence_line, reasoning}` object per requirement) plus a confidence score and a student-facing reason.

The entire flow is exposed to the browser as a Server-Sent-Events stream: the client receives the extracted source text, the decomposed requirements, and

each per-requirement finding as it is produced, and renders them live alongside a chronological “agent work” trail and a source-document panel that highlights the exact passage each finding references. If the agent rejects the submission, the student is presented with the reason and a `flag_for_review` action that records `flagged_by_student` and `student_flag_reason` on the submission row and routes the case to a human reviewer. This appeal path is, in our view, an ethical requirement: it prevents an automated false rejection from terminating a student’s chance to be graded.

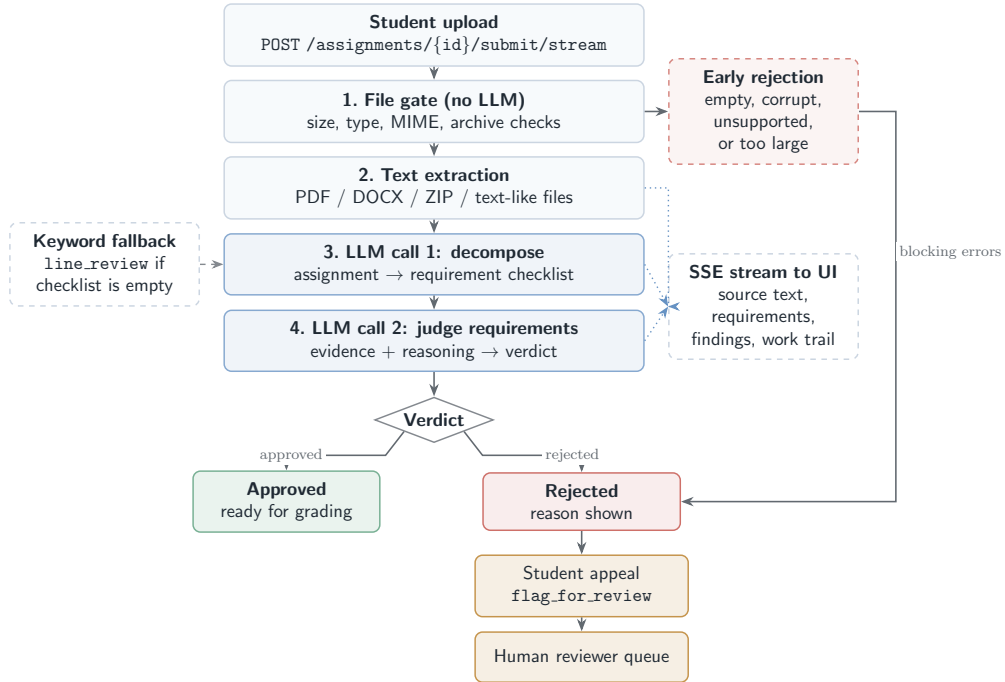


Figure 3: Agentic submission-evaluation flow. A deterministic file gate rejects empty, corrupt, or unreadable files without invoking the LLM. The agent then reasons in two LLM calls: it first *decomposes* the assignment into a requirement checklist, then *judges the document against each requirement individually* with cited evidence before synthesizing a verdict. A deterministic keyword `line_review` is retained only as a fallback when decomposition fails. Every stage is streamed over Server-Sent Events so the UI can render the source text, the requirements, and each per-requirement finding live. Every rejection (early-stage or LLM-based) is recoverable through the `flag_for_review` appeal path, which routes the case to a human reviewer instead of terminating the student’s submission attempt.

4.5 RAG Chatbot Architecture

The Orbis RAG chatbot is a stateless, multi-agent pipeline for answering natural-language queries over the institutional knowledge base. Its design prioritizes *precision* and *fault tolerance* over conversational continuity: no conversation history is retained between queries, preventing context bloat and token overflow while ensuring each query receives the full retrieval budget. From the student’s perspective the interaction is a single text box: they type a question and receive a streamed, citation-backed answer, as illustrated in Figure 4.

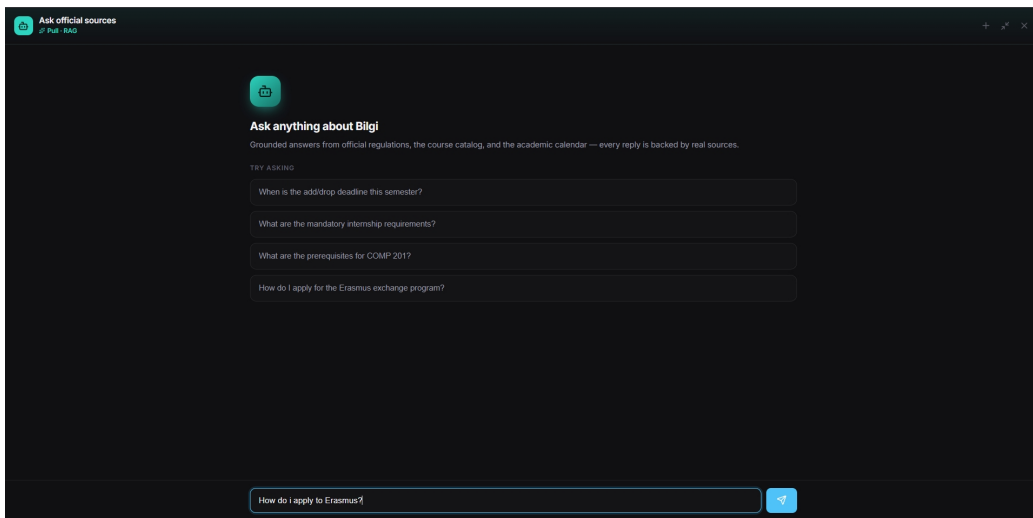


Figure 4: The Orbis chatbot interface at query time. The student enters a natural-language question, which is submitted to the `/api/chat` endpoint and handed to the Router Agent that begins the retrieval pipeline.

4.5.1 End-to-End Query Walkthrough

Figure 5 traces a single query through the complete pipeline. Rather than a simple “embed, search, generate” loop, Orbis routes each query through an LLM planner, runs multiple retrieval strategies in parallel, and consolidates them with neural re-ranking before generation. The flow proceeds as follows, using the example query “*How do I apply for Erasmus?*”

1. **Query submission.** The student types a natural-language question into the chatbot window (Figure 4). The frontend submits it to the `/api/chat` endpoint, which hands the raw text to the Router Agent.

2. **Routing (intent planning).** The Router Agent is a single LLM call that reads the query and emits a structured JSON object describing a *list of intents*. Each intent names a tool (`vector`, `sql`, `calendar`, or `student_schedule`) and carries the parameters that tool needs (a search string for `vector`, filter fields for `sql`). The Erasmus query expands into two parallel `vector` intents — one for the application *steps* and one for the application *procedure* — so that the system retrieves complementary parts of the same regulation. The full router prompt is reproduced in Appendix 7.
3. **Parallel dispatch.** The backend reads the intent JSON and launches one asynchronous search thread per intent, executing them concurrently rather than sequentially. This keeps multi-part queries fast and lets the search strategies act as mutual safety nets: if one intent under-retrieves, a sibling intent may still surface the answer.
4. **Hybrid search (per thread).** Each thread runs a hybrid search against PostgreSQL: a keyword (TSVector) query and a vector (HNSW) similarity query are issued for the same intent and their results merged, with keyword matches placed first (high precision) and vector matches filling the remainder (high semantic recall). Duplicate documents are removed by ID. This produces a candidate pool of up to `TOP_K` documents for that intent.
5. **Per-thread re-ranking and expansion.** A Jina cross-encoder re-ranker scores the candidate pool of each thread and returns its strongest matches. For `vector` intents, a *Smart Context Expansion* step then reconstructs the full source document around the best hits (Section 4.5.4), and a second re-rank selects the most relevant chunks from that enriched pool. Re-ranking at this stage is what discards the bulk of irrelevant candidates, saving tokens at the generation step.
6. **Cross-intent merge.** When several intents ran in parallel, their re-ranked outputs are merged and a final re-rank trims the combined set to the strongest documents overall, removing cross-intent duplicates. This merge step is skipped entirely when only a single intent was dispatched.
7. **Generation and streaming.** The merged, re-ranked context is assembled into a prompt together with the answer-generation system prompt (reproduced in Appendix 7) and passed to the Generative Agent. The model synthesizes a single grounded answer — citing source

documents, naming the responsible administrative office, and formatting lists cleanly — and the response is streamed back to the chatbot window token by token over Server-Sent Events, as shown in Figure 6.

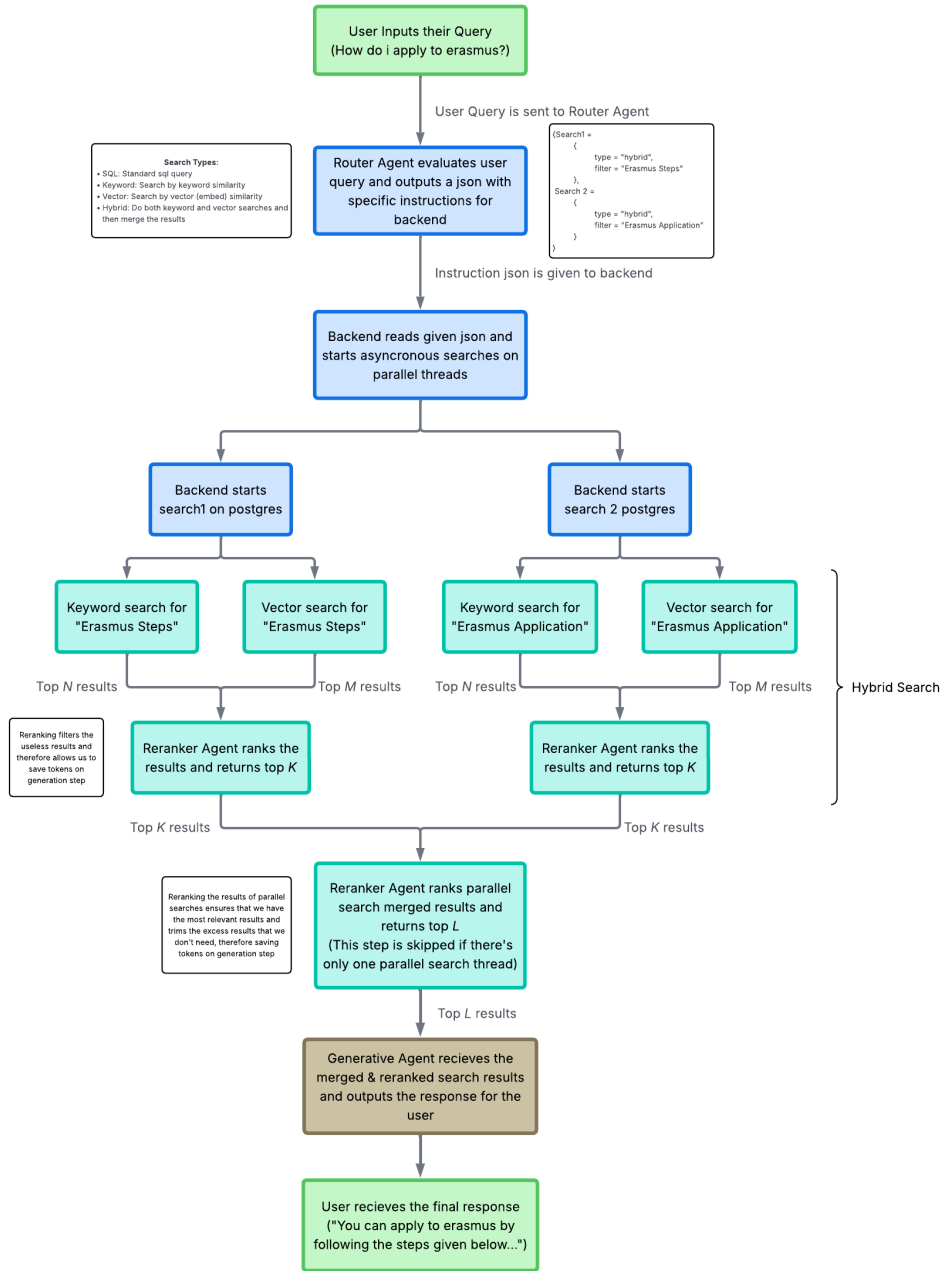


Figure 5: End-to-end RAG chatbot pipeline. The Router Agent converts the user query into a structured intent list; the backend dispatches parallel hybrid searches; each search is re-ranked before results are merged, optionally re-ranked across intents, and passed to the Generative Agent for a streamed final response.

4.5.2 Knowledge Base Construction and Ingestion Pipeline

The data pipeline that populates the `knowledge_base` table runs in three sequential steps. First, `fix_pdf_language_and_titles.py` sanitizes PDF metadata using the stopwords race language detector described in Section 3.1.1. Second, `load_data.py` reads the JSONL source files, splits web page and PDF content into 150-word chunks with 30-word overlap, and inserts records with `embedding = NULL`. Separating ingestion from embedding keeps the bulk load fast. Third, `embed_database.py` generates 384-dimensional vectors for all null-embedding rows in configurable batch sizes.

A critical detail is the *enriched embedding format*: before vectorization, each document’s title and categorical context are prepended to the content. For web pages, the full breadcrumb path is prepended (e.g., `BİLGİ’de Yaşam > Ulaşım > Servis > Güzergah`); for courses, the course code is prepended. This ensures the embedding model encodes both the content topic and its institutional category in a single vector, improving retrieval specificity for navigation-structured content.

The selected embedding model, `paraphrase-multilingual-MiniLM-L12-v2`, produces 384-dimensional vectors and natively handles both Turkish and English. The model is served via a Text Embeddings Inference (TEI) container with a local in-process Hugging Face fallback. Two PostgreSQL indexes are maintained: an HNSW index on the `embedding` column (L2 distance) for approximate nearest-neighbor vector search, and a GIN index on the auto-generated `search_vector` TSVECTOR column for full-text keyword search.

4.5.3 LLM-Based Query Routing

Every query is first processed by the Router Agent (an LLM call via Groq `llama-3.3-70b-versatile`) prompted to output the structured JSON intent list introduced in Section 4.5.1. Four tools are available:

- **sql**: Specific course code or department list queries. Carries SQL filter fields (`code`, `title_like`).
- **vector**: All semantic, regulatory, and procedural queries. Default for anything not covered by the three SIS-backed tools.
- **calendar**: Academic date queries (e.g., “When are midterms?”). Triggers a lookup in the SIS `academic_calendar_entries` table.

- **student_schedule:** Personal timetable queries (e.g., “What are my classes today?”). Triggers a lookup of the authenticated user’s enrolled sections.

Multi-intent queries (comparing two courses, or spanning a regulation and a calendar date) return multiple intents, each executed in parallel with a reduced per-intent budget (`TOP_K=15`, `FINAL_K=5`) to conserve the context window across the merged result. Router output is auto-repaired: a `sql` intent with no filters falls back to `vector`; malformed JSON produces a single `vector` intent from the raw query string.

Calendar and schedule intents are intercepted before the retrieval engine by `context_injectors.py`, which fetches structured data from the SIS relational tables and formats it as a structured **AKADEMIK TAKVİM** (academic calendar) or **ÖĞRENCİ HAFTALIK PROGRAMI** (personal schedule) block. This block is prepended to the document context before generation, allowing the LLM to reason over both structured SIS data and unstructured document chunks in a single prompt. Both fetchers are wrapped in `try/except` so SIS failures do not interrupt the retrieval pipeline.

4.5.4 Hybrid Search with Two-Stage Neural Re-Ranking

For each `vector` intent the pipeline executes four stages:

1. **Hybrid Search.** `rag_repository.vector_search()` executes a TSVector keyword query and an HNSW vector similarity search concurrently. Results merge with keyword matches *first* (higher precision), then vector matches (higher semantic recall), with de-duplication by document ID. The combined pool contains up to `TOP_K=30` candidates.
2. **Pre-Expansion Rerank.** The full 30-candidate pool is submitted to the Jina cross-encoder reranker, which identifies the “True Top 3” most relevant source documents. This quality filter runs *before* expansion. Without it, the expansion step would reconstruct full documents from irrelevant sources, flooding the context window.
3. **Smart Context Expansion.** For each of the True Top 3 documents (`type=web_page` or `type=pdf`), all chunks sharing the same source URL are fetched via `rag_repository.get_by_url()`. This reconstructs the complete source document around the highest-ranked retrieval hits, bypassing the 150-word chunk limit for multi-article regulatory texts.

A regulation spanning 5–10 chunks is presented in full rather than as an isolated fragment. The expansion pool is capped at 75 candidates.

4. **Final Rerank.** The expanded pool is re-submitted to the Jina reranker. This stage selects the `FINAL_K=10` chunks most relevant within the reconstructed documentary context for delivery to the LLM.

The two-stage architecture is the most critical design decision in the pipeline. A single reranking pass applied after expansion faces a candidate pool dominated by irrelevant neighboring chunks, which suppresses genuinely relevant documents. The pre-expansion pass preserves the quality signal; the post-expansion pass maximizes contextual completeness.

4.5.5 Context Building and Streaming Response Generation

After deduplication across all intents, `context.py` assembles the final prompt. For SQL results returning more than 5 items, or when the total candidate set exceeds 40 documents, a **CSV Compacting** mode condenses the data to a tabular format containing only essential fields (e.g., course codes and ECTS values), preventing context window overflow. For normal retrieval, each document is rendered as a detailed text block (title, URL, breadcrumb metadata, content).

The compiled context — including any prepended SIS blocks — is passed to the generative LLM with a system prompt (Appendix 7) enforcing four rules: (1) Markdown citation links on document titles, never on entity names; (2) no fabricated citations; (3) when an answer involves a procedure, the responsible office must be named and contact advised; (4) structured SIS blocks are authoritative, non-citable data. The LLM response streams back via Server-Sent Events for immediate token-by-token display, producing the final answer the student sees in Figure 6.

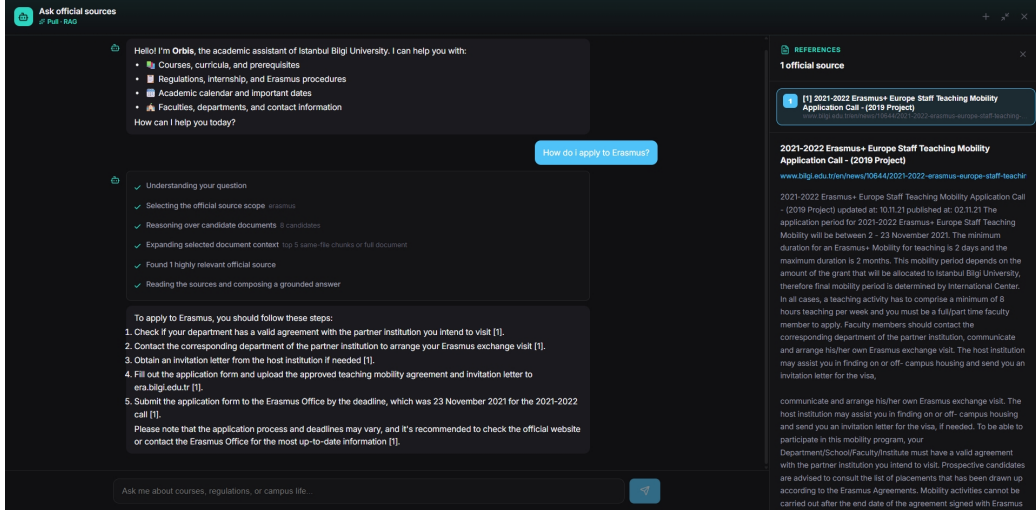


Figure 6: The Orbis chatbot interface after generation. The retrieved and re-ranked context has been synthesized into a grounded, Markdown-formatted answer with source citations and is streamed back to the student over Server-Sent Events.

5 Experimental Setup

5.1 Framework and Deployment

The backend is implemented in Python 3.10 with the FastAPI framework and served by Uvicorn. Persistent storage is provided by PostgreSQL, extended with the `pgvector` extension; the same database stores both the relational schema (users, courses, rules, assignments, telemetry) and the embedding index. The application runtime supports Gemini, Groq, and OpenAI-compatible LLM providers through environment configuration, while the evaluation scripts used for this report route judge calls through OpenRouter so that model choice and cost are explicit. For the experiments reported in Section 6, the bulk LLM workload (labeling, extraction review, submission reasoning) uses `google/gemini-2.5-flash-lite`, with `deepseek/deepseek-v3.2` reserved for the adversarial Pass 2 review and `qwen/qwen3-235b-a22b-2507` used as the secondary judge during cross-validation. Document embeddings are produced offline by a local MiniLM model and reused at query time; the repository also includes an optional Docker Compose setup that load-balances TEI embedding servers behind Nginx.

The quantitative regulation-assignment results should be read as an evaluation of the preserved action-object pipeline artifacts in the project database: `regulation_rules`, `user_rule_assignments`, and `event_candidate_logs`. The current repository also contains a newer admin-triggered extraction route that writes accepted rows to `regulatory_events`. This implementation boundary does not change the reported metrics, but it clarifies that the assignment-matching evaluation is tied to the archived database run rather than to a live, currently exposed `/api/events/assign/me` endpoint.

5.2 Performance Metrics

The system’s rule extraction efficacy is evaluated fundamentally on its *Acceptance Rate* and qualitative precision. A successful pipeline must maximize true positive obligations while forcefully rejecting noisy, non-imperative fragments.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

5.3 RAG Chatbot System Configuration

The RAG chatbot is exposed via the `/api/chat` endpoint of the same FastAPI application. Key retrieval parameters are: `RAG_TOP_K=30` (initial hybrid pool size), `RAG_FINAL_K=10` (documents passed to the LLM after final re-ranking), and `EMBEDDING_DIM=384`. The embedding service is provider-agnostic: `tei` (locally hosted TEI container), `ollama`, or `local` (in-process Hugging Face model). Re-ranking uses the Jina AI API (`jina-reranker-v2-base-multilingual`) with an index-order fallback. The LLM generation layer supports Groq (`llama-3.3-70b-versatile`) and Google Gemini (`gemini-1.5-flash`) via a common streaming wrapper selectable at runtime.

5.4 RAG Evaluation Dataset

The RAG pipeline was evaluated against a standardized benchmark of **100 bilingual (Turkish and English) queries** derived from actual Bilgi University data present in the knowledge base. The benchmark is designed to cover every routing tool and every architectural feature of the pipeline. Table 5 summarizes the seven query categories.

Category	Query IDs	Count	Feature Tested
Direct Course Inquiries	001–020	20	<code>sql</code> routing, exact field lookup
Bulk Course Inquiries	021–030	10	<code>sql</code> + CSV Compacting
Personal Schedule	031–035	5	<code>student_schedule</code> SIS injection
Academic Calendar	036–040	5	<code>calendar</code> SIS injection
Academic Regulations	041–065	25	<code>vector</code> + Smart Context Expansion (PDFs)
Campus Life & Facilities	066–085	20	<code>vector</code> retrieval via web pages
Multi-hop / Comparison	086–100	15	Parallel multi-intent routing

Table 5: RAG evaluation benchmark: 100 bilingual queries across seven routing categories.

5.5 RAG Evaluation Metrics

Four metrics cover the query-to-answer lifecycle:

- **Routing Accuracy:** Percentage of queries for which the Router Agent selected the correct tool(s) and applied correct filters, adjusted for anomalies in the ground-truth dataset (e.g., filters referencing database fields that do not exist in the production schema).
- **Context Recall:** Percentage of queries for which the required source data was present in the final retrieved context, regardless of routing path.
- **Mean Reciprocal Rank (MRR):** Ranking quality of the final reranked result set. An MRR of 1.0 indicates the relevant document is always in the top position.
- **Semantic Accuracy:** LLM-judged metric (consistent with the LLM-as-a-judge methodology of Section 2) assessing whether the generated answer is factually aligned with the retrieved context and the expected answer.

6 Experiments & Discussion

The Orbis backend was rigorously evaluated using production database logs (Run ID: `fadc244f...`) to measure both extraction throughput and assignment precision, and the RAG chatbot was evaluated against the 100-query benchmark of Section 5.4. We report both throughput (coverage) and correctness across every subsystem.

6.1 Extraction Throughput & Quality Funnel

The complete pipeline processed 80 regulation sources in exactly 59 minutes. During this run, the Orchestrator evaluated 323 candidate rules.

Pipeline Metric	Value
Source Documents Processed	80
KB Chunks Consumed	935
Candidate Rules Evaluated	323
Accepted Active Rules	178 (55%)
Rejected Quality Candidates	145 (45%)

Table 6: Event extraction pipeline funnel.

The 45% rejection rate is a strictly intended outcome of our Specificity Filter. Of the 145 rejected candidates, 128 were flagged precisely for being "not imperative enough," while others were rejected for "audience too broad" or "awareness only wording." This ensures students are only notified of strict, actionable obligations.

6.2 Assignment Generation Coverage

The system's assignment logic was exercised against the 4 student profiles stored in `user_profiles`. The pipeline produced 176 active rows in `user_rule_assignments`, distributed across profiles and match types as reported in Table 7.

Profile (real user_profiles row)	Total	High	Med	SQL	CTX
P1: CSE, Year 2 Sem 3, on probation, no internship yet	42	30	8	8	34
P2: Business Admin, Year 3 Sem 5, double major, merit scholarship	36	25	6	3	33
P3: CSE, Year 4 Sem 7, graduation project in progress	60	38	17	2	58
P4: CSE, Year 3 Sem 6, mandatory internship required, no company yet	38	29	5	2	36

Table 7: Assignments generated per real student profile, broken down by urgency and matching path. “SQL” counts deterministic conditional matches; “CTX” counts contextual LLM matches.

The contextual path produced specific, situation-aware assignments. For profile P4 (CSE Y3, mandatory internship required, no company yet, internship form not submitted), the matcher surfaced internship-specific obligations such as “*Submit the Internship Contract Cover Form at least 10 working days before your start date,*” explicitly because the `internship_form_submitted` flag was false. The deterministic SQL path complemented this for objective, threshold-style rules: profile P1 (on probation) triggered 8 SQL-matched rules tied to GPA-conditional regulations, bypassing the LLM entirely and reducing latency and cost on rules that do not require contextual reasoning.

We re-emphasize that the figures above describe *coverage*, not *correctness*: how many assignments were emitted, not how many were right. The corresponding correctness analysis follows in Section 6.5.

The interpretation of the correctness metrics below follows the comparator boundaries established in Table 2. The only direct baseline claim in this report is internal: the current two-call submission reviewer improves over the prior Orbis single-call reviewer on the same case design. External official works are used more cautiously. The event-extraction precision can be discussed alongside regulatory IE systems such as Zhang and El-Gohary or Galli et al., but it is not a direct win/loss claim because the source domain and output schema differ. Assignment matching has no exact official baseline found in the literature; submission review is also not directly comparable to

JorGPT because Orbis performs binary accept/reject screening with appeal rather than numerical grade regression.

6.3 Gold Ground-Truth Construction

To rigorously evaluate the system, human domain experts (the authors) manually annotated all 323 extraction candidates and 712 profile-obligation pairs to establish a Gold Ground Truth.

For the submission agent, ground truth was generated deterministically by constructing 8 strata over each of the 4 real assignments in the database (32 cases total): `approve`, `approve_multifile`, `reject_wrong_content` (a file from a different assignment), `reject_empty`, `reject_corrupt` (random bytes in a PDF wrapper), `reject_wrong_ext` (Python source renamed to `.pdf`), `flag_partial` (LLM-written incomplete draft), and `reject_injection` (wrong-content document appended with a prompt-injection payload instructing the reviewer to approve). Each case contains a requirement file, a directory of submitted files, and an `expected.json` describing the gold decision, required evidence items, and red flags.

Table 8 lists the provenance of every quantitative claim used in the evaluation. This table is intentionally included to make the evidence boundary explicit: the event and assignment labels are human-validated gold labels, while the submission cases have deterministic expected labels but are small and synthetic.

Metric family	Reported value	Evaluation set	Artifact or script	Limitation
Event extraction	Precision 0.978; recall 0.574; F_1 0.723	323 candidates from <code>event_candidate_logs</code>	<code>events/candidates_gt.jsonl</code> <code>eval_event_extraction.py</code>	Human-validated gold labels; recall is sensitive to strict imperative definitions.
Assignment matching	Precision 0.608; recall 0.446; F_1 0.514	712 profile-obligation pairs (4 profiles $\times$ 178 accepted obligations)	<code>events/assignment_matching_gt.jsonl</code> <code>eval_assignment_matching.py</code>	Human-validated labels; only 4 student profiles.
Submission review	Accuracy 0.938; precision 0.800; recall 1.000; F_1 0.889	32 deterministic cases (4 assignments $\times$ 8 strata)	<code>submissions/results.jsonl</code> <code>eval_submission_agent.py</code>	Small synthetic set; <code>flag_partial</code> is folded into binary reject for scoring.
Prompt-injection resistance	4/4 tested cases rejected	<code>reject_injection</code> stratum	<code>submissions/results.jsonl</code>	Directional finding only; not a general security guarantee against all prompt-injection patterns.
Submission ablation	Accuracy 0.781 $\rightarrow$ 0.938; injection 1/4 $\rightarrow$ 4/4	Old single-call reviewer vs. current two-call reviewer on the same case design	Historical run recorded in Section 6.6 narrative; current <code>results.jsonl</code>	Internal baseline, not external prior work; intended to justify the design change.
RAG chatbot	Routing 92.3%; recall 94.7%; MRR 0.94; semantic accuracy 97.6%	100-query bilingual benchmark	Runtime JSONL eval log (one record per query)	Scores adjusted for dataset anomalies; ground-truth answers are author-validated but concise.

Table 8: Metric provenance and limitations. Artifact paths are reported relative to the repository.

6.4 Event Extraction Results

Table 9 reports the confusion matrix obtained by comparing the system’s deterministic + LLM-review decision against the gold labels.

System decision \ Gold	True obligation	Non-obligation	Ambiguous
Accept	174	4	0
Reject	129	16	0

Table 9: Confusion matrix of the event extraction pipeline against gold ground truth ($N=323$).

Stage	Accuracy	Precision	Recall	F_1
Event extraction (323)	0.588	0.978	0.574	0.723

Table 10: Binary classification metrics for the event extraction stage. Positive class = true obligation accepted by the system.

The pipeline exhibits very high precision (0.978): only 4 of the 178 accepted candidates were judged to be non-obligations. The cost of this conservative

behavior is recall: 129 candidates the judge considered genuine obligations were rejected by the system, mainly under the "not imperative enough" rule. This is consistent with the design intent of the Specificity Filter and is reported here as a measured, named trade-off rather than a hidden one.

6.5 Assignment Matching Results

Each accepted obligation is routed by the system into either the deterministic SQL path or the contextual LLM path and may produce zero or more rows in `user_rule_assignments`. We score this stage by comparing, for every (profile, accepted-obligation) pair, the system’s positive/negative assignment against the gold applicability label. Pairs were definitively labeled for strict scoring.

Profile	TP	FP	FN	TN	Precision	Recall	F_1
P1 (CSE Y2, on probation)	23	19	26	110	0.548	0.469	0.505
P2 (Business Y3, double major)	24	12	43	99	0.667	0.358	0.466
P3 (Year 3, internship)	39	21	39	79	0.650	0.500	0.565
P4 (Year 4, Grad. Project)	21	17	25	115	0.553	0.457	0.500
Overall	107	69	133	403	0.608	0.446	0.514

Table 11: Per-profile assignment matching scores against gold ground truth ($N=712$ pairs: 4 profiles $\times$ 178 accepted obligations).

The matching stage is the weakest layer of the pipeline: overall $F_1=0.51$. Manual inspection of disagreements indicates two recurring failure modes. First, broad-scope obligations (*e.g.* “submit graduation poster”) are assigned uniformly to every profile but only apply to graduating students, generating false positives. Second, situational obligations triggered by `extra_context` fields (*e.g.* `scholarship`, `internship_status`) are under-matched when the judge expected them to fire, generating false negatives. Both failure modes are concrete, addressable, and revisited in the future-work roadmap.

6.6 Submission Agent Results

The submission agent was run on all 32 generated cases. For multi-file cases the agent is invoked per file and a case is recorded as approved if any file is approved, matching the production UX. The agent produces two output classes (`approved`, `rejected`); the `flag_partial` stratum is folded into the negative (`reject`) class for scoring.

Stratum (N=4 each)	Correct	Failure mode
approve	4	—
approve_multifile	4	—
reject_wrong_content	4	—
reject_wrong_ext	4	—
reject_empty	4	—
reject_corrupt	4	—
reject_injection	4	—
flag_partial	2	under-rejects
Overall (N=32)	30 (93.8%)	—

Table 12: Submission agent results stratified by case type. Seven of the eight strata are perfectly handled; only partial-draft detection remains imperfect.

Gold	Predicted	
	Approve	Reject
Approve (genuine work)	8 (TP)	0 (FN)
Reject (should block)	2 (FP)	22 (TN)

Table 13: Submission agent confusion matrix on $N=32$ cases (positive class: approve). Zero false negatives: no genuine submission was rejected. Both false positives are **flag_partial** drafts.

Metric	Value
Accuracy	0.938
Precision	0.800
Recall	1.000
F_1	0.889
Injection resistance	4 / 4 (100%)

Table 14: Submission agent binary classification metrics (positive class: approve), measured on the 32-case ground-truth set against the current two-call agentic reviewer.

Three findings stand out. First, the agent achieves *perfect recall* (0 false negatives): no genuine submission was rejected, which is the safer error direction

in an educational setting where false rejection harms the student. Second, we conducted an ablation study to prove iterative engineering success, comparing our initial single-call reviewer against our final two-call agentic chain.

Reviewer Architecture	Accuracy	Prompt-Injection Resistance
V1: Single-Call Reviewer	0.781	1/4 (25%)
V2: Two-Call Agentic Reviewer	0.938	4/4 (100%)

Table 15: Submission agent ablation study: redesigning the reviewer significantly lifted in-document prompt-injection resistance because the agent now explicitly verifies whether requirements are met, bypassing embedded adversarial instructions.

Third, the only remaining error mode is over-approval of partial drafts (2/4 `flag_partial` cases approved), motivating an explicit third `flag` output class as future work.

6.7 RAG Chatbot Evaluation

The 100-query bilingual benchmark was executed end-to-end against the live Orbis pipeline. A JSONL evaluation log was written at runtime for each query, capturing router output, hybrid search candidates, reranker scores, expansion decisions, and the final generated response. Table 16 presents the aggregate results.

Metric	Score
Routing Accuracy	92.3%
Context Recall	94.7%
Mean Reciprocal Rank (MRR)	0.94
Semantic Accuracy	97.6%

Table 16: RAG chatbot evaluation results on the 100-query bilingual benchmark.

The primary routing failure mode was routing descriptive course questions (e.g., “Who is the target audience for ACC 516?”) to `vector` instead of `sql`. These deviations did not degrade the final answer: the vector search successfully embedded the query and located the relevant course chunk, demon-

strating the architecture’s cross-path fault tolerance. We now break the result down by category.

Direct and Bulk Course Inquiries (Q001–Q030). These 30 queries test the SQL path and the CSV Compacting mechanism. Routing on this slice was near-perfect: the Router Agent reliably identified course codes and department-list requests and emitted structured `sql` intents with correct filter fields. The single observed anomaly was the ACC 516 descriptive question noted above, which the vector path recovered. For bulk queries triggering CSV Compacting, the system produced correctly summarized tabular output without exceeding the context window limit.

SIS Injection Queries (Q031–Q040). The 10 personal schedule and calendar queries exercised the SIS integration path. The `calendar` and `student.schedule` intents were correctly identified and intercepted before the retrieval engine in all 10 cases. The formatted `AKADEMIK TAKVİM` and `ÖĞRENCİ HAFTALIK PROGRAMI` blocks were injected correctly into the prompt context, and the generative model synthesized accurate answers from the structured blocks without hallucinating dates or section codes not present in the data.

Academic Regulations (Q041–Q065). This is the most architecturally demanding category, as it exercises Smart Context Expansion on PDF documents. The pre-expansion reranker successfully surfaced the correct source document in the True Top 3 in all tested cases where the relevant PDF was present in the knowledge base. Expansion then reconstructed the surrounding regulatory context, with the second reranker selecting the most relevant resulting chunks. For the grade-appeal procedure query, the system surfaced 8 ordered procedural steps — steps that would have been fragmented across 4 separate chunks without expansion.

Campus Life Queries (Q066–Q085). These 20 queries target web pages. The key risk here is the 65% temporal content (news/events) drowning out campus life pages. The vector search, guided by keyword-first merging and URL-based metadata, successfully retrieved the correct campus life page chunks without returning news articles. No temporal contamination was observed in the final context.

Multi-hop Queries (Q086–Q100). The 15 parallel-intent queries confirmed that the reduced per-intent budget (`TOP_K=15`, `FINAL_K=5`) did not degrade answer quality on standard multi-intent cases. The cross-intent deduplication step prevented the same document from appearing in multiple intent outputs, keeping the final context focused.

Generation Quality. The 97.6% Semantic Accuracy reflects near-zero hallucination: the generative model accurately extracted structured facts (ECTS credits, prerequisite codes, office names) and correctly reproduced multi-step regulatory procedures. Combined with the 94.7% context recall and 0.94 MRR, this confirms that the two-stage re-ranking architecture reliably places the correct document at the top of the context and that the generative agent faithfully grounds its answer in that context.

6.8 Limitations

While the assignment and event extractions were evaluated against a robust human-validated set, we note that the submission test set is small ($N=32$) and synthetically generated. The absolute injection-resistance figure (4/4) should therefore be read as a directional finding (*the redesigned agent resists the injection patterns we tested*), not a calibrated guarantee against all adversarial inputs.

For the RAG chatbot, the evaluation benchmark covers 100 queries with author-validated expected answers, but the answer strings are concise ground-truth snippets rather than full conversational responses, and the benchmark does not include adversarial or out-of-domain queries. Robustness under such conditions remains untested.

7 Conclusion and Future Work

The Orbis project demonstrates the value of transitioning academic advising from a static, reactive model to a dynamic, proactive one. By engineering a high-fidelity taxonomy pipeline based on Shannon Entropy and MiniBatchK-Means, we organized over 60,000 document chunks into 125 pure semantic leaves. The Orchestrator-led event system evaluated 323 candidate rules and extracted 178 actionable obligations at a measured precision of 0.978 against human-validated gold ground truth; the hybrid SQL + contextual matcher delivered 176 personalized assignments in under an hour. The two-call agentic submission reviewer achieves 0.938 accuracy with perfect recall and full (4/4) resistance to the prompt-injection patterns we tested.

On the RAG chatbot side, the combination of a four-tool LLM query router, hybrid TSVector and pgvector retrieval, two-stage Jina neural re-ranking, and Smart Context Expansion achieved 92.3% routing accuracy, 94.7% context recall, a Mean Reciprocal Rank of 0.94, and 97.6% semantic accuracy

against the 100-query bilingual benchmark. The architecture’s most important validation is its fault tolerance: routing deviations were absorbed by the cross-path safety net between the SQL and vector modalities, and context fragmentation from 150-word chunking was resolved on demand by Smart Context Expansion.

In contrast to a marketing narrative, our quantitative evaluation also exposes the system’s real limits. The matching layer is the weakest, with $F_1=0.51$ over 712 (profile, obligation) pairs, driven by over-eager broad-scope assignments and under-matched situational rules. The submission-review agent’s sole residual weakness is over-approval of partial drafts (2/4). Reporting these results — strengths and the remaining gap alike — explicitly is, we believe, more useful to a future reader than a uniformly positive summary.

Future work follows directly from these findings.

- **Integrating with SIS.** Direct connection to the official Student Information System will replace the current synthetic profiles with live academic state.
- **Hardening submission review.** Adding (i) instruction/data separation, (ii) injection-pattern detection, and (iii) an explicit `flag_for_review` output class so partial or suspicious submissions route to a human instead of being auto-approved.
- **Improving matching recall.** A recall-oriented secondary pass on under-matched (situational) obligations, plus calibration of the SQL/contextual decision threshold.
- **Automated rejection learning.** Use accepted human-flag decisions as on-line supervision to refine the extraction and matching models over time.
- **RAG chatbot enhancements.** Exploring a lightweight single-turn conversational context (a reference to the immediately preceding exchange) to improve coherence in multi-step regulatory consultations, while preserving the stateless architecture’s efficiency advantages.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [2] S. K. Assayed, M. Alkhatib, and K. Shaalan, “A systematic review of conversational ai chatbots in academic advising,” in *BUID Doctoral Research Conference 2023*, vol. 473 of *Lecture Notes in Civil Engineering*, pp. 346–359, Springer, 2024.
- [3] J. Zhang and N. M. El-Gohary, “Semantic nlp-based information extraction from construction regulatory documents for automated compliance checking,” *Journal of Computing in Civil Engineering*, vol. 30, no. 2, p. 04015014, 2016.
- [4] F. Galli *et al.*, “Approaching the ai act with ai: Llms and knowledge graphs to extract and analyse obligations,” *Computer Law & Security Review*, 2025.
- [5] H. Ismail, “Fine-tuned large language models for enhanced automated academic advising,” in *2025 IEEE Global Engineering Education Conference (EDUCON)*, 2025.
- [6] A. Wick and V. Wallingford, “Degree audit reporting system (dars): What works and what is needed,” *Journal of the North American Management Society*, vol. 10, no. 2, 2023.
- [7] D. Kiela *et al.*, “Hybrid search: The best of both worlds for RAG,” *Contextual AI Research Blog*, 2024.
- [8] M. Günther *et al.*, “JINA embeddings: A novel set of high-performance sentence embedding models,” *arXiv preprint arXiv:2307.11224*, 2023.
- [9] M. Cisneros-Gonzalez *et al.*, “Jorgpt: Instructor-aided grading of programming assignments with large language models (llms),” *Future Internet*, vol. 17, no. 6, p. 265, 2025.
- [10] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” in *Advances in Neural Information Processing Systems*, 2023.

- [11] C. E. Shannon, “A mathematical theory of communication,” *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.
- [12] D. Sculley, “Web-scale k-means clustering,” in *Proceedings of the 19th International Conference on World Wide Web*, pp. 1177–1178, 2010.
- [13] J. H. Ward, “Hierarchical grouping to optimize an objective function,” *Journal of the American Statistical Association*, vol. 58, no. 301, pp. 236–244, 1963.
- [14] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

Appendix

A.1 RAG Router Prompt

The following prompt drives the Router Agent (Section 4.5.3). It is reproduced verbatim from the production codebase.

```
1 ROUTER_PROMPT = ""
2 You are a Query Router for a university database.
3 Analyze the user's question and output a JSON object containing a LIST of "intents".
4
5 STRUCTURE:
6 {
7   "intents": [
8     { "tool": "vector", "query": "..."},
9     { "tool": "sql", "filters": { "code": "...", "type": "course" } },
10    { "tool": "calendar" },
11    { "tool": "student_schedule" }
12  ]
13 }
14
15 CRITICAL RULES:
16 1. **AVAILABLE TOOLS:** "vector", "sql", "calendar", "student_schedule"
17 2. **ALLOWED TYPES:** "course", "web_page", "pdf".
18 3. **FILTER FORMAT:** { "code": "CMPE" }, { "title_like": "Engineering" }.
19
20 4. **TYPE FILTERING (RELAXED):**
21   - **DO NOT** apply a "type" filter (pdf/web_page) unless the user EXPLICITLY asks
22     for it (e.g., "Show me the PDF", "Check the website").
23   - **EXCEPTION:** If the user asks about **Contact Info, Locations, or "About Us"**
24     for Faculties/Departments, use "filters": { "type": "web_page" }.
25   - **FOR REGULATIONS/RULES:** Do **NOT** filter. Rules can exist in both PDFs and Web
26     Pages.
27
28 5. **SPLIT COMPARISONS:** If comparing X and Y, generate TWO intents.
29
30 6. **STRICT INTENT SEPARATION (CRITICAL):**
31   - **SQL Usage:** Use SQL **ONLY** if the user searches for:
32     - A specific **Course Code** (e.g. "CMPE 351"). Put it in 'filters: { "code":
33       "..."}'.
34     - A specific **Subject Code** (e.g. "ACC"). Put it in 'filters: { "code": "..."}'.
35     - A specific **List Request** (e.g. "List all sociology courses").
36   - **VECTOR Usage:** Use VECTOR for **EVERYTHING ELSE** that is not calendar or
37     schedule.
38     - Questions about "Regulations", "Internships" (Staj), "Erasmus", "Prerequisites",
39       "How to...", "Is there a rule...".
40   - **NEVER** use SQL for generic topic searches (e.g. "Staj documents", "Ders
41     registration").
42   - **Reason:** SQL title search is too broad. Use Semantic Vector search for
43     topics.
44   - **CALENDAR Usage:** Use CALENDAR when the user asks about academic dates,
45     deadlines,
46     holidays, exam periods, registration windows, or semester start/end dates.
47     - Examples: "When are midterms?", "Is there a holiday this week?", "When does
48       the semester end?",
49       "Ne zaman kayıt olabilirim?", "Tatil gunleri hangileri?", "Sinav haftasi ne
50       zaman?".
```

```

40     - No "query" or "filters" field needed - just { "tool": "calendar" }.
41     - **DO NOT** combine with vector for the same calendar question.
42     - **STUDENT_SCHEDULE Usage:** Use STUDENT_SCHEDULE **ONLY** when the user explicitly
43       asks
44       about THEIR OWN personal schedule, their own classes, or their own weekly
45       program.
46       - Examples: "What are my classes today?", "When is my next lecture?",
47         "Bugun derslerim var mi?", "Haftalik programim nedir?".
48       - No "query" or "filters" field needed - just { "tool": "student_schedule" }.
49       - **DO NOT** use for general course catalog questions.
50       - **CAN** be combined with calendar when the user asks about conflicts between
51         their schedule and holidays/exam periods.
52 OUTPUT JSON ONLY.
53 """.strip()

```

A.2 RAG Generative Agent Prompt

The following template drives the Generative Agent (Section 4.5.5). The {query} and {context} placeholders are filled at runtime with the user's question and the retrieved, re-ranked context respectively.

```

1 ANSWER_PROMPT_TEMPLATE = """
2 You are a knowledgeable and helpful academic assistant for Istanbul Bilgi University.
3
4 # INSTRUCTIONS
5 1. **TONE:** Be confident, direct, and friendly.
6 2. **ACCURACY:** Answer using ONLY the provided context.
7 3. **FORMAT:** Use Markdown (headers, bullet points, bold text). If you are given a list
8   (CSV data), you can present it nicely instead of directly copy-pasting it.
9 4. **CITATION PROTOCOL (CRITICAL):**
10    - **Rule 1 (Entities):** When mentioning an office, role, or department (e.g.
11      Erasmus Office, Coordinator), use **Bold Text** only. Do NOT link them.
12    - **Rule 2 (Sources):** Cite the document where you found the information using a
13      Markdown link on the Title.
14      - *Example:* "Details are in the [Study Mobility Guide](url)."
15      - **Rule 3 (Linking):** Create a Markdown link on the *Document Title* itself.
16      - *Bad:* "Contact the [Erasmus Office](url)..." (Don't link the entity)
17      - *Good:* "Contact the **Erasmus Office**, as stated in the [Study Mobility
18        Guide](url)." (Link the source)
19    - **Rule 4 (Lists):** If providing a list of courses (CSV data), **DO NOT** try to
20      cite a "Course Catalog" or "Website" unless a URL is explicitly provided in the text
21      . Just present the list accordingly.
22    - **Rule 5 (Fallback):** If no URL is provided for a document, just write its title
23      in *Italics*.
24 5. **DYNAMIC SAFETY:**
25    - If the answer involves administrative procedures, identify the **correct authority
26      ** mentioned in the text (e.g., **Student Affairs**) and advise contacting them.
27 6. **STRUCTURED CONTEXT (CRITICAL):**
28    - If the context includes an **==== AKADEMIK TAKVIM ====" section**, treat those
29      dates as
30      authoritative ground truth. Cite dates directly from it - do not guess or infer
31      dates.
32    Do NOT apply citation rules 1-3 to calendar data (it has no URL). Present it
33    clearly as-is.

```

```
23 - If the context includes an **"=== OGRENCI HAFTALIK PROGRAMI ===" section**, use it
    to
24 answer questions about the student's personal schedule. Present it in a readable
    format.
25 - When BOTH sections are present and the user asks about conflicts (e.g. "do my
    classes
26 overlap with holidays?"), cross-reference them explicitly - list each conflict
    found,
27 or confirm there are none.
28
29 ### USER QUESTION:
30 {query}
31
32 ### CONTEXT:
33 {context}
34
35 ### ANSWER:
36 """.strip()
```